

Documentation technique - Vite & Gourmand

1. Vue d'ensemble

Vite & Gourmand est une application PHP rendue côté serveur. Un contrôleur frontal reçoit toutes les requêtes, choisit un contrôleur métier, puis charge une vue dans le gabarit commun.

Navigateur

- > public/index.php
- > Controller
- > Model / Service
- > MySQL ou MongoDB
- > View
- > app/Views/layouts/main.php
- > Réponse HTML

Le code cible PHP 8.2 et utilise `declare(strict_types=1)`.

2. Architecture MVC

Contrôleur frontal

`public/index.php` :

- démarre une session sécurisée ;
- charge Composer, `.env`, la base et le service CSRF ;
- pose les en-têtes de sécurité ;
- associe `?page=...` à une action ;
- transforme les `HttpException` en pages 4xx ;
- journalise les erreurs inattendues et renvoie une page 500 ;
- prépare les horaires et badges de navigation ;
- charge le gabarit principal.

Contrôleurs

Classe	Responsabilité
HomeController	accueil et avis validés
MenuController	catalogue, détail et gestion des menus
DishController	gestion des plats
AuthController	inscription, connexion, profil et mot de passe
OrderController	commandes client et employé
ReviewController	dépôt et modération des avis
AdminController	comptes employés et statistiques
HoursController	horaires
ContactController	demandes de contact
PageController	mentions, CGV et confidentialité

Les contrôleurs héritent de `App\Core\Controller`, qui centralise le rendu, les redirections, les erreurs et les contrôles d'accès.

Modèles

Les modèles SQL héritent de `App\Models\Model` et reçoivent une instance PDO. `Review` et `OrderAnalytics` utilisent le pilote officiel MongoDB.

Les vues n'exécutent pas de requête et n'appliquent pas de règle métier.

Services

Service	Rôle
DishManagementService	validation des plats et suppression protégée
MenuManagementService	validation, composition et gestion des menus
MenuFilterService	normalisation et recherche asynchrone des menus
OrderPricing	calcul déterministe du prix
OrderWorkflowService	transitions, notifications et e-mails de statut
PasswordPolicy	règle commune de robustesse des mots de passe

Service	Rôle
UserRegistrationService	validation et création d'un client
MailService	contenu et envoi des e-mails
csrf.php	création du champ et validation du jeton
Environment	chargement structuré du fichier <code>.env</code>
Url	URL absolues des pages et ressources

3. Routage et accès

Le routeur est une table explicite :

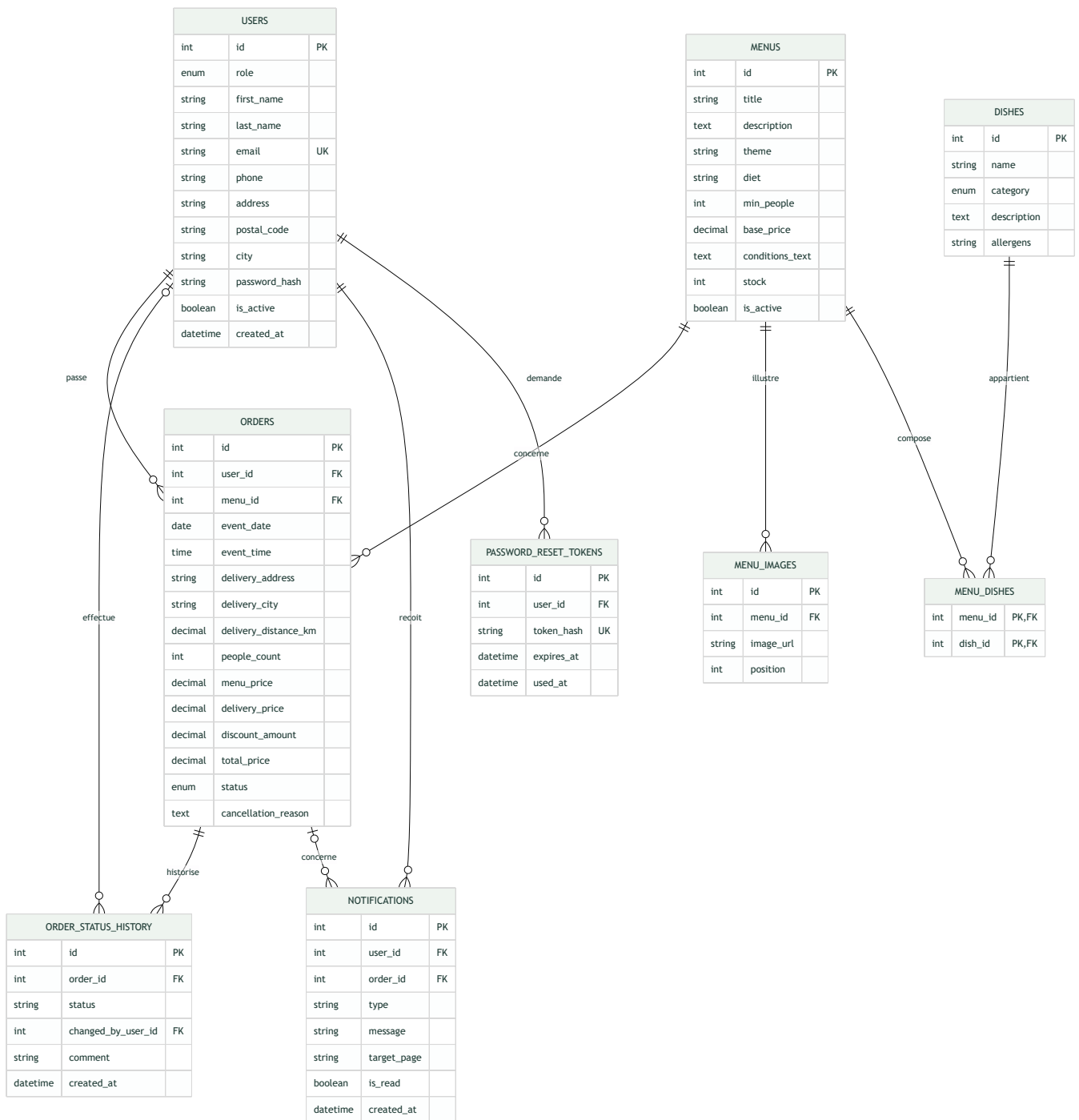
```
'employee-orders' => [OrderController::class, 'manage'],
'employee-menus' => [MenuController::class, 'manage'],
'admin-dashboard' => [AdminController::class, 'dashboard'],
```

Contrôles :

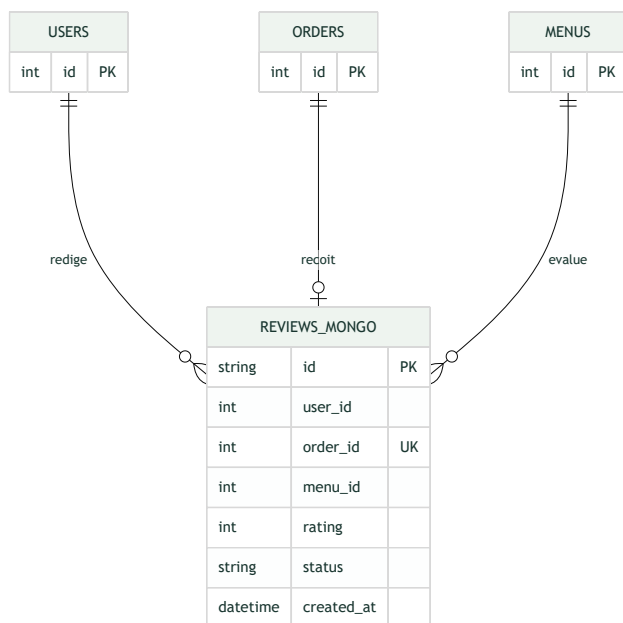
- `requireUser()` : compte connecté, encore actif et resynchronisé avec la base ;
- `requireRole(['employee', 'admin'])` : back-office opérationnel ;
- `requireRole(['admin'])` : statistiques et employés.

Une autorisation est toujours vérifiée côté serveur. Le masquage d'un lien de navigation n'est jamais considéré comme une protection.

4. Modèle conceptuel relationnel



Collection active et tables indépendantes



CONTACT_MESSAGES		
int	id	PK
string	full_name	
string	email	
string	subject	
text	message	
boolean	is_read	
datetime	created_at	

BUSINESS_HOURS		
int	day_of_week	PK
string	day_label	
time	first_open	
time	first_close	
time	second_open	
time	second_close	
boolean	is_closed	

CONTACT_MESSAGES et **BUSINESS_HOURS** n'ont pas de relation avec une autre table. **REVIEWS_MONGO** représente la collection active : ses identifiants SQL sont des références applicatives et non des clés étrangères MongoDB.

Table complémentaire historique :

- **reviews** SQL : table historique non utilisée par la fonctionnalité active.

Le schéma complet est dans `database/schema.sql`. Les migrations additives pour une ancienne installation sont regroupées dans `database/migrations/`.

5. Données NoSQL

Collection `reviews`

```
{
  "id": "review_...",
  "order_id": 42,
  "user_id": 7,
  "user_name": "Prénom Nom",
  "menu_id": 3,
  "menu_title": "Menu Classique",
  "rating": 5,
  "comment": "Commentaire du client",
  "status": "pending",
  "created_at": "2026-07-18 12:00:00"
}
```

Statuts : `pending` , `validated` , `refused` .

L'index unique `unique_review_per_order` empêche deux avis pour une même commande. Les index sont décrits dans `database/mongodb-indexes.js` .

Collection `order_analytics`

Cette projection contient l'identifiant de commande, le menu, le montant, le statut et la date. Le tableau de bord l'actualise par `upsert` , puis agrège :

- le nombre de commandes non annulées par menu ;
- le chiffre d'affaires par menu ;
- les résultats limités par menu et période.

Si MongoDB est indisponible, le contrôleur calcule les mêmes données depuis SQL et affiche explicitement la source de secours.

6. Règles métier

Prix

Pour un menu de minimum `M` , de prix de base `P` , commandé pour `N` personnes :

```
prix_menu = P × (N / M)
remise = 10 % du prix_menu si N >= M + 5
livraison = 0 € à Bordeaux, sinon 5 € + (distance_km × 0,59 €)
total = prix_menu - remise + livraison
```

Le calcul JavaScript donne un retour immédiat, mais le serveur recalcule toujours le total avec `OrderPricing`.

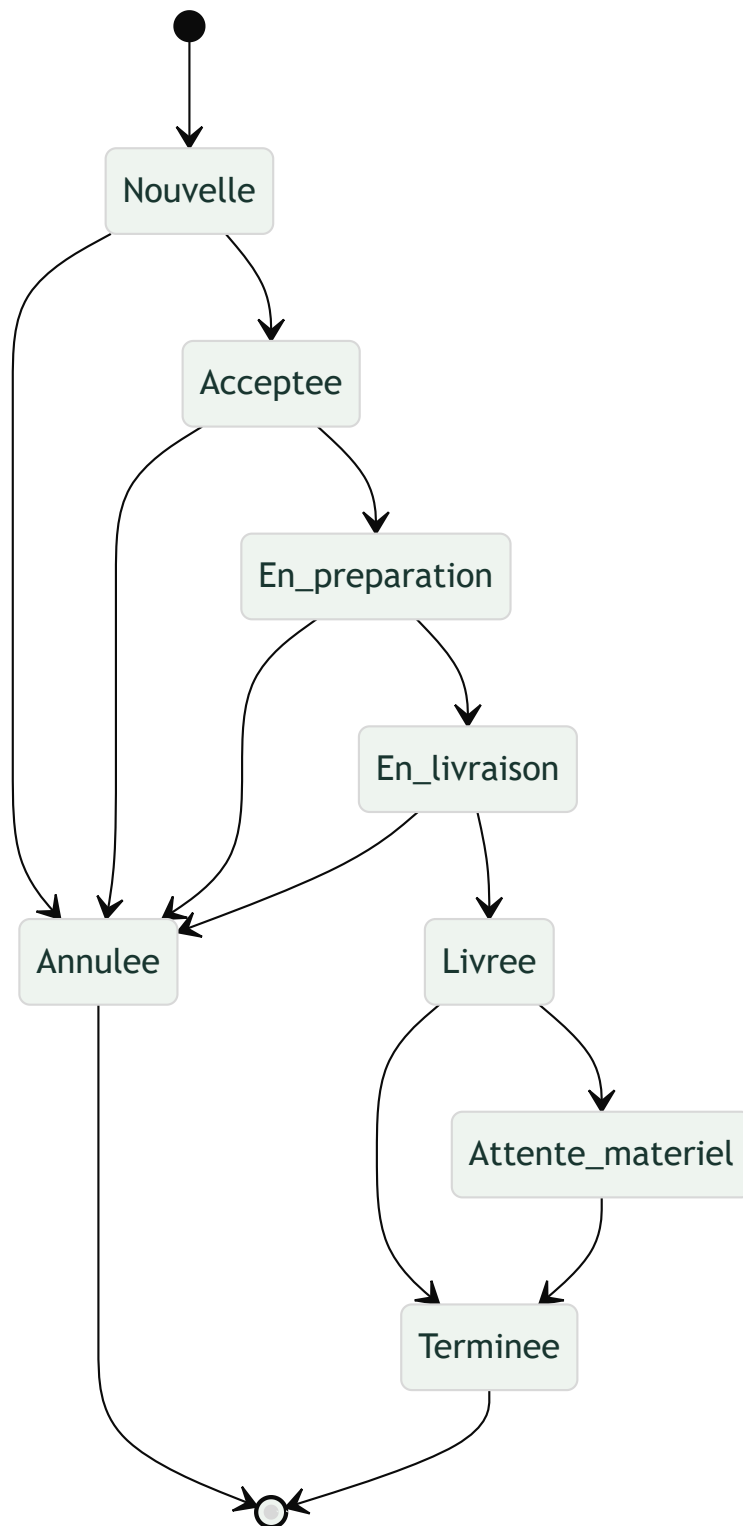
Stock

La création d'une commande :

1. démarre une transaction ;
2. verrouille le menu avec `SELECT ... FOR UPDATE` ;
3. contrôle activité et stock ;
4. insère la commande ;
5. décrémente le stock ;
6. crée le premier historique ;
7. valide la transaction.

Une annulation restaure le stock une seule fois.

Cycle de commande

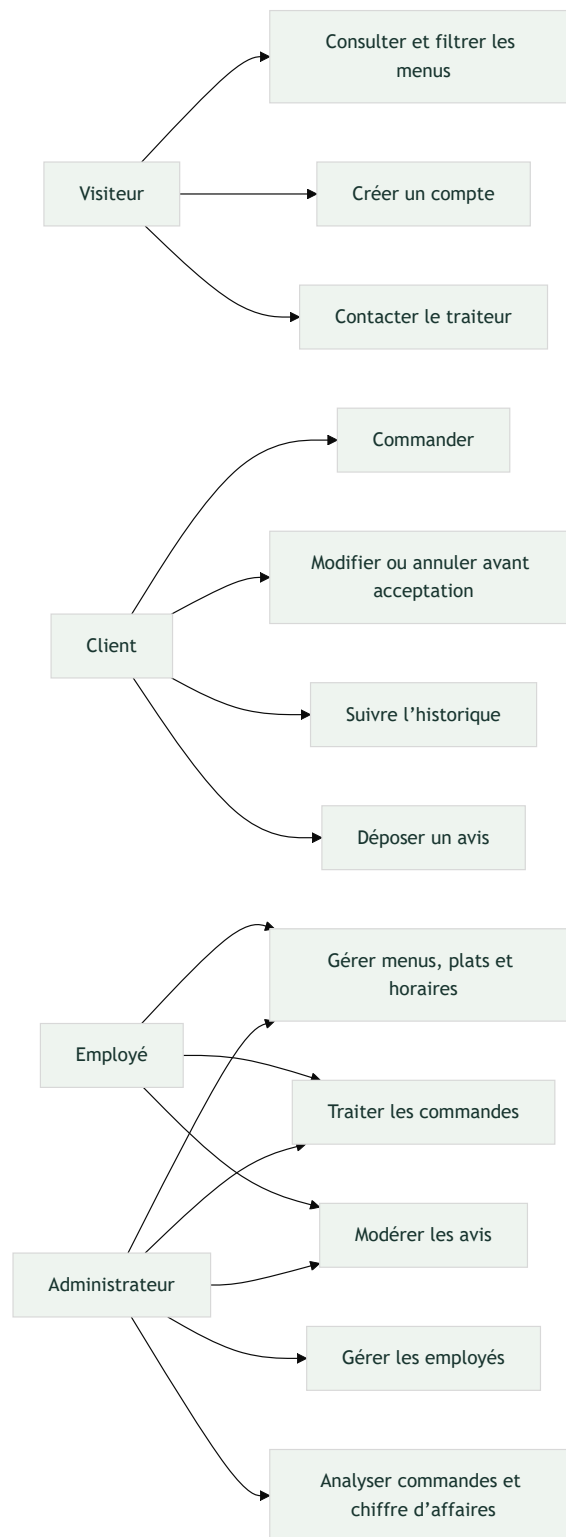


Le client modifie ou annule uniquement au statut `nouvelle` . L'employé doit indiquer un moyen de contact et un motif pour annuler.

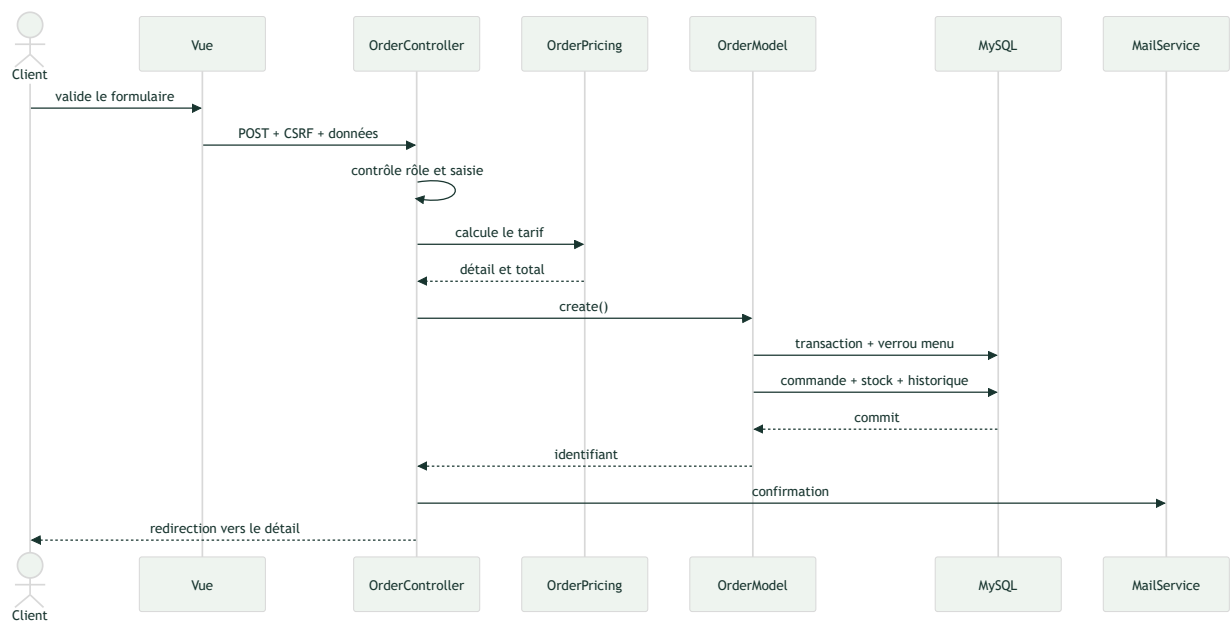
Menus

- titre de 3 à 150 caractères ;
- description et conditions obligatoires ;
- minimum et stock entiers positifs ;
- prix positif ;
- une entrée, un plat et un dessert au minimum ;
- six images maximum ;
- archivage logique avec `is_active = 0` et `stock = 0`.

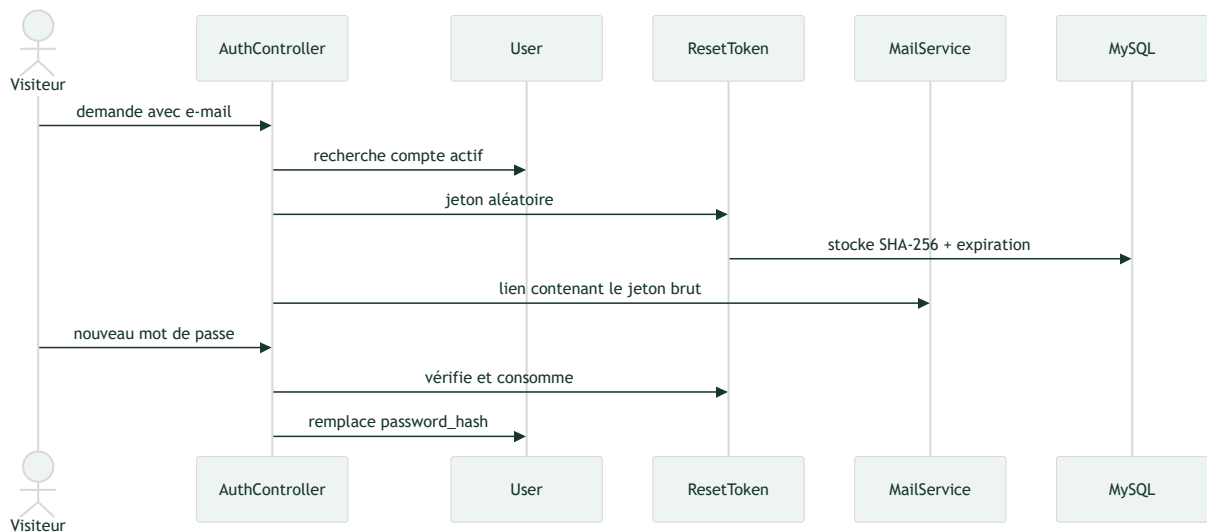
7. Cas d'utilisation



8. Séquence de commande



9. Séquence de réinitialisation



10. Sécurité

Menace	Mesure
Injection SQL	requêtes préparées PDO
XSS réfléchi ou stocké	<code>htmlspecialchars</code> dans les vues
CSRF	jeton de session et POST sur les actions

Menace	Mesure
Vol de mot de passe	<code>password_hash</code> / <code>password_verify</code>
Énumération de comptes	réponse neutre au mot de passe oublié
Brute force	cinq échecs puis blocage de quinze minutes
Fixation de session	<code>session_regenerate_id(true)</code> à la connexion
Accès horizontal	recherche commande avec <code>user_id</code> du compte
Accès vertical	<code>requireRole</code> côté contrôleur
Secret Git	fichiers réels ignorés et exemples neutralisés
Fuite d'erreur	détails masqués en production, journalisation serveur
Clickjacking / MIME	en-têtes <code>X-Frame-Options</code> et <code>nosniff</code>
Chargement de contenu tiers	politique CSP, ressources front-end locales
Repli vers HTTP	HSTS lorsque l'application est servie en HTTPS

Les jetons de mot de passe sont générés par `random_bytes`, stockés uniquement sous forme de hachage SHA-256 et invalidés après utilisation. `PasswordPolicy` impose de 10 à 72 caractères avec majuscule, minuscule, chiffre et caractère spécial avant tout nouveau hachage.

11. Résilience

- l'accueil reste disponible si MongoDB ne répond pas ;
- l'espace client conserve le suivi des commandes sans les avis ;
- l'administration utilise SQL si les statistiques MongoDB échouent ;
- un échec de `mail()` est journalisé sans annuler la transaction métier ;
- les horaires ont des valeurs par défaut si leur lecture échoue ;
- Bootstrap et Chart.js sont locaux, sans dépendance CDN.

12. Front-end

Bootstrap fournit la grille, les formulaires, alertes et composants accessibles.

`public/css/style.css` définit l'identité et le responsive.

JavaScript reste progressif :

- `menu-filters.js` interroge `?page=api-menus` avec `fetch`, puis affiche les cartes correspondantes sans rechargement ;
- `app.js` masque les confirmations, confirme les actions risquées, gère les horaires, l'annulation employé et l'aperçu du prix.

Le serveur n'accorde jamais sa confiance au calcul ou au filtrage du navigateur.

13. Tests

```
composer test
composer test:all
composer test:integration
composer validate --no-check-publish
```

- `mvc_architecture.php` contrôle classes, routes et vues ;
- `domain_rules.php` contrôle prix, validations, transitions et URL ;
- `security_accessibility.php` contrôle CSRF, labels, champs et images ;
- `database_integration.php` contrôle les transactions sur données temporaires.

La recette HTTP vérifie les pages publiques, les accès par rôle, la page 404 et l'absence d'avertissement PHP. Les captures Playwright couvrent accueil, menus et détail sur ordinateur (`1440 × 1000`) et mobile (`390 × 844`).

14. Configuration et déploiement

Voir `déploiement.md`. Les secrets sont placés dans `config/database.php`, `config/mongodb.php` et, si nécessaire, `.env` ou les variables serveur équivalentes. Ces fichiers ne doivent jamais être commités.

15. Limites connues

- `mail()` dépend de la configuration de l'hébergeur et doit être testé en production ;
- les mentions légales doivent recevoir l'identité définitive de l'éditeur ;
- la conformité RGAA complète nécessite un audit spécialisé ;
- MongoDB Atlas doit autoriser l'IP du serveur de production.